



Trabajo Práctico N°3: Transformada Rápida de Fourier

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Abril de 2026

Resumen

Realizar un programa aplicativo que sea capaz de implementar la Transformada Rápida de Fourier (FFT) sobre un buffer de memoria de tamaño configurable (512, 1024 y 2048 muestras), cuyos datos son adquiridos a partir del sistema desarrollado en el Laboratorio N°1, contemplando además las distintas frecuencias de muestreo disponibles (8 kHz, 16 kHz, 24 kHz, 40 kHz y 48 kHz). El sistema permitirá habilitar o deshabilitar la aplicación de la FFT mediante una tecla de la placa de evaluación, funcionando en modo procesamiento o bypass. El resultado de la FFT será transmitido a través del puerto serie (SDU) para su posterior visualización en una PC mediante una herramienta gráfica, donde se representará el espectro de la señal.

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
2. Marco teórico	4
2.1. Análisis espectral de señales	4
2.2. Transformada Discreta de Fourier (DFT)	4
2.3. Transformada Rápida de Fourier (FFT)	4
2.4. Resolución espectral y parámetros de análisis	5
2.5. Implementación en sistemas embebidos	5
2.6. Biblioteca CMSIS-DSP	5
3. Placa de Desarrollo y Herramientas Utilizadas	6
3.1. Microcontrolador STM32F446RE	6
3.2. Periféricos clave utilizados	7
3.2.1. ADC (Analog-to-Digital Converter)	7
3.2.2. DAC (Digital-to-Analog Converter)	7
3.2.3. DMA (Direct Memory Access)	7
3.2.4. TIM (Timers)	8
3.2.5. GPIO – General Purpose Input/Output	8
3.2.6. Herramientas de desarrollo	8
4. Diseño del programa	9
5. Programa principal	9
5.1. Configuración de periféricos	9
5.1.1. Configuración del ADC	9
5.1.2. Configuración del DAC	10
5.1.3. Configuración del temporizador	10
5.1.4. Configuración del DMA	10
5.1.5. Configuración de GPIO	11
5.1.6. Inicialización de los filtros digitales	11
5.1.7. Ejecución en tiempo real	11
6. Resultados experimentales	11
7. Conclusiones	11

1. Introducción

El análisis espectral de señales constituye una herramienta fundamental dentro del procesamiento digital de señales (DSP), ya que permite representar una señal en el dominio de la frecuencia y extraer información relevante sobre su contenido armónico. En sistemas embebidos, la implementación eficiente de este tipo de análisis resulta clave en aplicaciones como procesamiento de audio, monitoreo de vibraciones, telecomunicaciones y sistemas de diagnóstico.

En este trabajo práctico se aborda la implementación de la Transformada Rápida de Fourier (FFT) sobre una plataforma basada en el microcontrolador STM32F446RE, continuando con la arquitectura desarrollada en los laboratorios anteriores. El sistema permite adquirir señales analógicas mediante el ADC, procesarlas en tiempo real y analizar su espectro de frecuencias utilizando buffers de distinto tamaño (512, 1024 y 2048 muestras), lo que introduce un compromiso entre resolución espectral y carga computacional.

Para garantizar el funcionamiento en tiempo real, se mantiene el uso de técnicas eficientes como transferencias mediante DMA en modo circular y procesamiento por bloques, minimizando la intervención del CPU. Asimismo, se emplea la biblioteca CMSIS-DSP, optimizada para arquitecturas ARM Cortex-M, que permite implementar la FFT de manera eficiente en formato de punto fijo.

El sistema desarrollado permite habilitar o deshabilitar el procesamiento mediante una tecla de usuario (modo FFT o bypass), y transmitir los resultados a través de una interfaz serie hacia una PC, donde se visualiza el espectro de la señal. Esto posibilita analizar el impacto de parámetros clave como la frecuencia de muestreo y el tamaño de la ventana sobre la resolución en frecuencia, la aparición de aliasing y las limitaciones prácticas del sistema.

De esta manera, el trabajo no solo busca implementar la FFT en un entorno embebido, sino también comprender los compromisos inherentes al análisis espectral en tiempo real, integrando conceptos teóricos con su aplicación práctica sobre hardware real.

1.1. Objetivos del proyecto

- Implementar la Transformada Rápida de Fourier (FFT) sobre señales adquiridas en tiempo real utilizando el ADC del microcontrolador STM32F446RE.
- Procesar buffers de distinto tamaño (512, 1024 y 2048 muestras), analizando el impacto en la resolución espectral y el rendimiento del sistema.
- Integrar el procesamiento de la FFT dentro de la arquitectura basada en DMA y doble buffering, garantizando operación en tiempo real.
- Utilizar la biblioteca CMSIS-DSP para la implementación eficiente de la FFT en formato de punto fijo (Q15).
- Permitir la habilitación o deshabilitación del procesamiento (modo FFT/bypass) mediante una entrada por GPIO.
- Transmitir los resultados de la FFT a través de una interfaz serie (SDU) para su posterior visualización en una PC.
- Analizar el espectro de la señal obtenida, evaluando el efecto de la frecuencia de muestreo y el tamaño de la ventana sobre la resolución en frecuencia.

2. Marco teórico

2.1. Análisis espectral de señales

El análisis espectral consiste en representar una señal en el dominio de la frecuencia, permitiendo identificar las componentes sinusoidales que la conforman. Mientras que en el dominio temporal se observa la evolución de la señal en el tiempo, el dominio frecuencial permite analizar su contenido armónico, lo cual resulta fundamental en aplicaciones de procesamiento de audio, comunicaciones y sistemas de medición.

Para señales digitales, este análisis se realiza a partir de muestras discretas en el tiempo, obtenidas mediante un proceso de muestreo. La precisión del análisis espectral depende directamente de parámetros como la frecuencia de muestreo y la cantidad de muestras consideradas.

2.2. Transformada Discreta de Fourier (DFT)

La Transformada Discreta de Fourier (DFT) es una herramienta matemática que permite transformar una señal discreta en el tiempo $x[n]$ en su representación en frecuencia $X[k]$. Se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi \frac{kn}{N}}, \quad k = 0, 1, 2, \dots, N-1$$

donde N es la cantidad de muestras de la señal. Cada valor $X[k]$ representa la contribución de una componente de frecuencia específica dentro de la señal.

El principal inconveniente de la DFT es su alto costo computacional, ya que requiere del orden de $O(N^2)$ operaciones, lo cual la vuelve poco eficiente para aplicaciones en tiempo real sobre sistemas embebidos.

2.3. Transformada Rápida de Fourier (FFT)

La Transformada Rápida de Fourier (FFT) es un algoritmo optimizado para calcular la DFT de manera eficiente, reduciendo la complejidad computacional a $O(N \log_2 N)$. Esto la hace viable para implementaciones en tiempo real.

El algoritmo más común es el de tipo Cooley-Tukey, el cual se basa en dividir recursivamente la DFT en transformadas más pequeñas.

El procedimiento general consiste en:

- Reordenar las muestras de entrada según el esquema de inversión de bits (bit-reversal).
- Dividir el problema en etapas, donde en cada una se combinan pares de muestras mediante operaciones conocidas como *butterflies*.
- Aplicar factores de rotación complejos (twiddle factors), definidos como:

$$W_N^k = e^{-j2\pi \frac{k}{N}}$$

A lo largo de $\log_2 N$ etapas, se obtiene finalmente el espectro completo de la señal.

2.4. Resolución espectral y parámetros de análisis

La resolución en frecuencia de la FFT está dada por:

$$\Delta f = \frac{f_s}{N}$$

donde f_s es la frecuencia de muestreo y N el número de muestras. Esto implica que:

- A mayor tamaño de ventana (N), mayor resolución en frecuencia.
- A mayor frecuencia de muestreo, mayor ancho de banda analizado.

Sin embargo, existe un compromiso entre resolución espectral y tiempo de procesamiento, ya que buffers más grandes implican mayor carga computacional y latencia.

Además, en implementaciones prácticas pueden aparecer fenómenos como:

- **Aliasing**: debido a una frecuencia de muestreo insuficiente.
- **Fuga espectral (spectral leakage)**: causada por el truncamiento de la señal en ventanas finitas.
- **Resolución limitada**: cuando las componentes espectrales están muy cercanas entre sí.

2.5. Implementación en sistemas embebidos

En sistemas embebidos, la FFT se aplica sobre bloques de datos adquiridos en tiempo real. Para ello, es común utilizar buffers circulares y técnicas de doble buffering, permitiendo procesar un bloque mientras el siguiente se está adquiriendo.

El uso de aritmética en punto fijo (como formato Q15) permite mejorar el rendimiento en microcontroladores sin unidad de punto flotante, reduciendo el tiempo de cómputo y el consumo de recursos.

2.6. Biblioteca CMSIS-DSP

La biblioteca CMSIS-DSP proporciona funciones optimizadas para procesamiento digital de señales en microcontroladores ARM Cortex-M. Incluye implementaciones eficientes de la FFT tanto en punto flotante como en punto fijo (Q15 y Q31).

Para el cálculo de la FFT en formato Q15, se utilizan funciones como:

- `arm_rfft_q15`: para señales reales.
- `arm_cfft_q15`: para señales complejas.

Estas funciones requieren la inicialización previa de una estructura de configuración, la cual contiene parámetros como el tamaño de la FFT y los factores de rotación pre-computados.

El flujo típico de implementación es:

1. Conversión de las muestras del ADC al formato Q15.
2. Inicialización de la estructura de la FFT.
3. Ejecución de la FFT sobre el buffer de entrada.
4. Cálculo de la magnitud del espectro (por ejemplo, mediante `arm_cmplx_mag_q15`).

Estas rutinas están altamente optimizadas y aprovechan las instrucciones SIMD del núcleo Cortex-M4, permitiendo una ejecución eficiente en aplicaciones de tiempo real.

3. Placa de Desarrollo y Herramientas Utilizadas

El proyecto fue desarrollado sobre la placa STM32 Nucleo-F446RE, basada en el microcontrolador STM32F446RE, perteneciente a la familia STM32F4 de STMicroelectronics. Esta plataforma integra una arquitectura avanzada ARM Cortex-M4, periféricos orientados a procesamiento de señales, y herramientas de desarrollo que permiten implementar sistemas en tiempo real de forma eficiente.

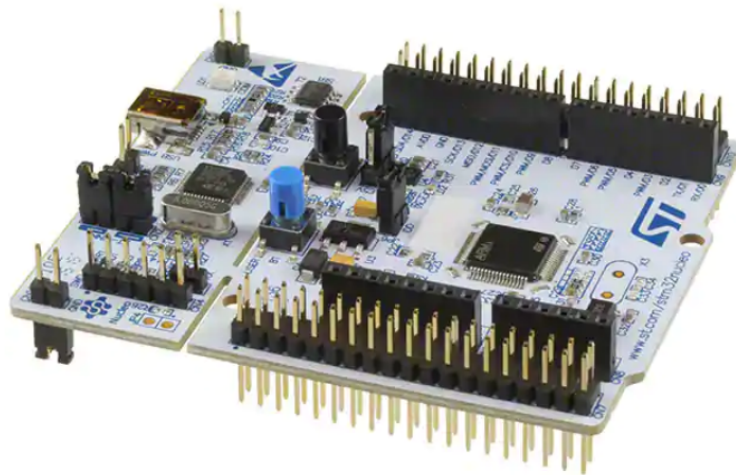


Figura 1: Placa de desarrollo

3.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de 32 bits basado en el núcleo ARM Cortex-M4, optimizado para procesamiento digital con soporte de instrucciones DSP y unidad de punto flotante opcional (FPU simple precisión). Sus principales características relevantes al proyecto son:

- Frecuencia de operación: hasta 180MHz
- Memoria Flash: 512KB
- RAM: 128KB
- Tensión de operación: 1.8V a 3.6V
- Paquete: LQFP64 (compatible con la Nucleo-F446RE)

3.2. Periféricos clave utilizados

3.2.1. ADC (Analog-to-Digital Converter)

El STM32F446RE cuenta con 3 ADCs independientes (ADC1, ADC2 y ADC3), cada uno de 12 bits de resolución y con capacidad de hasta 2.4 MSPS (mega muestras por segundo) en modo intercalado. Para este proyecto se utiliza el ADC1 en modo regular y con DMA activado. Sus características técnicas relevantes incluyen:

- Resolución seleccionable: 6, 8, 10 o 12 bits (se usa 12 bits para mayor precisión).
- Tasa de muestreo máxima (single conversion): hasta 2.4 MSPS.
- Rango de entrada: 0V a VREF (típicamente 3.3V).
- Tiempo de muestreo configurable: desde 3 hasta 480 ciclos de ADC.
- Trigger externo: el inicio de la conversión puede ser disparado por un temporizador, lo cual permite muestrear a frecuencias precisas.
- Modo DMA: transferencia automática de los resultados a un buffer en memoria (sin intervención del CPU).
- Modo continuo + DMA circular: permite muestreo ininterrumpido con mínimo overhead.

3.2.2. DAC (Digital-to-Analog Converter)

El STM32F446RE incluye 2 canales DAC (DAC1 y DAC2), ambos de 12 bits, con capacidad de salida en modo bufferizado (con baja impedancia) o sin buffer (alta velocidad). Se utiliza DAC1 en este proyecto para convertir las muestras digitales de salida nuevamente en una señal analógica. Características clave:

- Resolución de conversión: 12, 10 u 8 bits.
- Velocidad típica: hasta 1 MSPS (modo sin buffer).
- Modo bufferizado: reduce el ruido, ideal para salidas de audio.
- Trigger externo: puede sincronizarse con un temporizador (por ejemplo TIM6/TRGO).
- Soporte DMA: se permite la alimentación directa de datos desde memoria al DAC sin CPU.
- Salidas disponibles: a través de pines analógicos dedicados (por ejemplo, PA4 para DAC1).

3.2.3. DMA (Direct Memory Access)

El controlador DMA2 es utilizado para las transferencias entre ADC/DAC y RAM. El STM32F446RE cuenta con dos controladores DMA (DMA1 y DMA2), con un total de 16 canales (streams) y soporte para múltiples periféricos. Características específicas:

- Transferencia en background, sin intervención del CPU.
- Modo circular: ideal para buffers cíclicos o doble buffering.

- Prioridad programable por canal.
- Transferencia por palabras (16 bits en Q15).
- Eventos por mitad de buffer y buffer completo: permite usar doble buffer sincronizado.

3.2.4. TIM (Timers)

El STM32F446RE cuenta con 14 temporizadores (4 avanzados, 2 básicos, 8 generales), todos con configuraciones de prescaler, auto-reload y posibilidad de generar eventos de actualización. Se utilizan para generar señales periódicas que actúan como trigger de muestreo. Características clave:

- TIM2-TIM5: de 32 bits, con amplio rango de conteo (útiles para precisión en frecuencias bajas).
- Configuración de frecuencia: vía Prescaler y ARR (Auto-Reload Register).
- Trigger de ADC/DAC: mediante señal TRGO (Trigger Output).
- Modo master-slave: permite sincronización entre periféricos.

3.2.5. GPIO – General Purpose Input/Output

Los pines GPIO permiten la interacción con el usuario mediante botones (teclas) y LEDs. Se utilizan:

- Velocidad de salida seleccionable: Low, Medium, Fast, High.
- Pull-up/Pull-down configurables.
- Interrupciones por flanco ascendente, descendente o ambos.

3.2.6. Herramientas de desarrollo

Para la implementación del proyecto se utilizaron las siguientes herramientas:

- STM32CubeIDE: entorno de desarrollo oficial de STMicroelectronics, basado en Eclipse, que permite configurar periféricos gráficamente, generar código y compilar con GCC.
- STM32CubeMX: herramienta para la configuración inicial de pines, relojes, periféricos y generación de código base.
- CMSIS y CMSIS-DSP: bibliotecas proporcionadas por ARM para facilitar el uso de instrucciones de bajo nivel y rutinas DSP optimizadas en punto fijo.
- GDB y ST-Link: utilizados para depuración, profiling y medición de tiempos mediante DWT->CYCCNT.

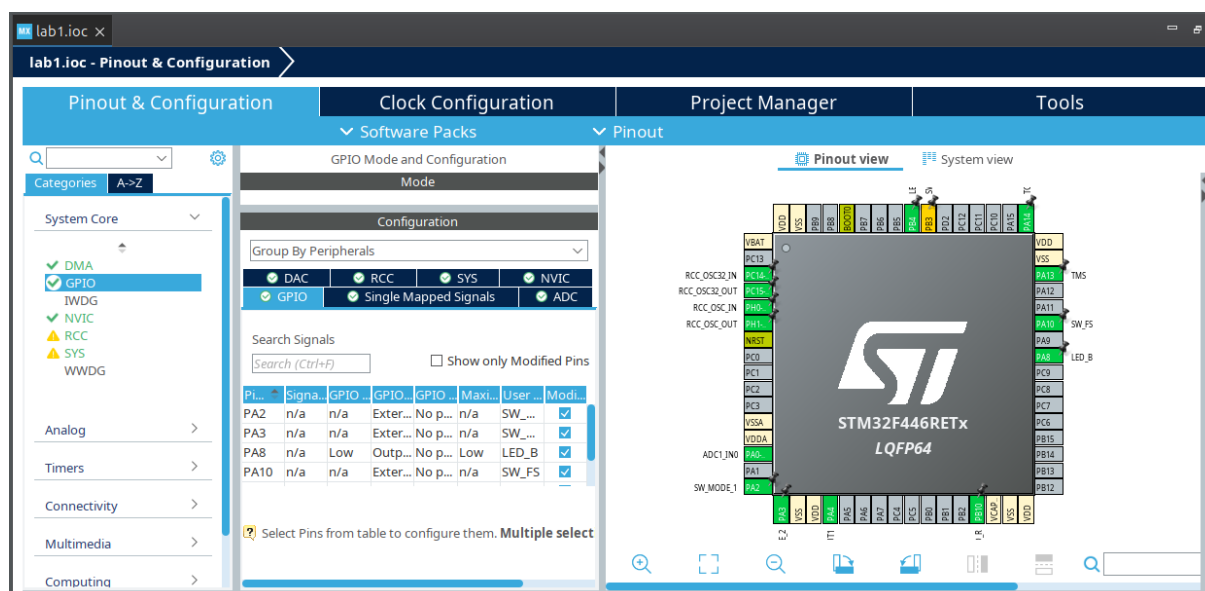


Figura 2: Entorno de desarrollo STM32CUBE IDE

4. Diseño del programa

5. Programa principal

5.1. Configuración de periféricos

El programa principal inicializa los periféricos necesarios para realizar la adquisición, procesamiento y reproducción de la señal en tiempo real. La arquitectura general mantiene el mismo esquema del Trabajo Práctico N^o1, incorporando ahora la etapa de filtrado digital mediante la biblioteca CMSIS-DSP.

La inicialización de los periféricos se realiza mediante las funciones generadas automáticamente por STM32CubeMX:

- MX_GPIO_Init()
- MX_DMA_Init()
- MX_ADC1_Init()
- MX_TIM2_Init()
- MX_DAC_Init()

Cada uno de estos periféricos cumple una función específica dentro del sistema de procesamiento digital en tiempo real.

5.1.1. Configuración del ADC

El convertor analógico-digital (ADC1) se utiliza para adquirir la señal de entrada con una resolución de 12 bits. El ADC es disparado periódicamente por el temporizador TIM2, lo que garantiza una frecuencia de muestreo precisa.

La transferencia de datos se realiza mediante DMA en modo circular, permitiendo almacenar continuamente las muestras en un buffer sin intervención directa del CPU.

La inicialización del ADC en modo DMA se realiza con:

```
1  HAL_ADC_Start_DMA(&hadc1,  
2      (uint32_t*) adc_buffer ,  
3      BUFFER_SIZE);
```

El uso de DMA permite implementar un esquema de doble buffer, donde la mitad del buffer puede procesarse mientras la otra mitad continúa llenándose con nuevas muestras.

5.1.2. Configuración del DAC

El convertor digital-analógico (DAC1) se utiliza para reconstruir la señal procesada y permitir su visualización mediante un osciloscopio.

El DAC también opera en modo DMA, lo que permite enviar continuamente los datos procesados desde memoria hacia la salida analógica sin intervención del CPU.

La inicialización del DAC se realiza mediante:

```
1  HAL_DAC_Start_DMA(&hdac ,  
2      DAC_CHANNEL_1 ,  
3      (uint32_t*) dac_buffer ,  
4      BUFFER_SIZE ,  
5      DAC_ALIGN_12B_R);
```

5.1.3. Configuración del temporizador

El temporizador TIM2 se utiliza como base de tiempo para definir la frecuencia de muestreo del sistema. Este temporizador genera eventos periódicos (TRGO) que disparan simultáneamente el ADC y el DAC, garantizando sincronización entre adquisición y reconstrucción de la señal.

La frecuencia de muestreo puede modificarse dinámicamente mediante interrupciones externas, ajustando el valor del registro Auto-Reload del temporizador.

El temporizador se inicia mediante:

```
1  HAL_TIM_Base_Start(&htim2);
```

Los valores de autoreload utilizados para cada frecuencia de muestreo son:

- 8 kHz
- 16 kHz
- 22 kHz
- 44 kHz
- 48 kHz

5.1.4. Configuración del DMA

El controlador DMA permite transferir datos entre los periféricos y la memoria sin intervención del CPU. En este sistema se utiliza para:

- Transferir muestras desde el ADC hacia el buffer de entrada
- Transferir muestras procesadas hacia el DAC

El uso de DMA reduce la carga del procesador y permite garantizar procesamiento en tiempo real.

5.1.5. Configuración de GPIO

Los pines GPIO se utilizan para interactuar con el usuario mediante pulsadores y LEDs.

Las interrupciones externas permiten:

- Cambiar la frecuencia de muestreo
- Cambiar el tipo de filtro (LP, HP, BP, BS)
- Activar el modo bypass del filtro

Además, los LEDs RGB indican visualmente el estado del sistema y la frecuencia de muestreo seleccionada.

5.1.6. Inicialización de los filtros digitales

Luego de inicializar los periféricos, se configuran las estructuras necesarias para implementar los filtros digitales en formato Q15 utilizando la biblioteca CMSIS-DSP.

La inicialización de los filtros se realiza mediante:

```
1  arm_fir_init_q15(&FPB,  
2      FPB_8K_NUM_TAPS ,  
3      FPB_8K_COEFFS ,  
4      FPB_status ,  
5      BUFFER_SIZE / 2);
```

Se definen filtros paso bajo, paso alto, pasa banda y rechazo de banda para cada frecuencia de muestreo soportada por el sistema.

5.1.7. Ejecución en tiempo real

Una vez inicializados los periféricos, el sistema comienza a operar en tiempo real mediante interrupciones del DMA. El procesamiento de la señal se realiza dentro de las funciones callback:

- HAL_ADC_ConvHalfCpltCallback()
- HAL_ADC_ConvCpltCallback()

Estas funciones procesan cada mitad del buffer utilizando los filtros digitales seleccionados.

6. Resultados experimentales

7. Conclusiones